SE_45: Frontend

I built a website that I have had in mind for a while now and had worked on before. At first it was just a simple travel app where everyone can post the trip they had and review them and say what they liked. I then got the idea of making it more of something where you can track your expenses and add a budget to it and then post it. I built a React application using Vite, Tailwind CSS, and React Router v7, this made managing expenses and tracking budgets much easier.

So what I built is user authentication flow with registration, login, and protected routes, a dashboard with the trips overview and some small statistics, expense tracking with category selection and date validation, an analytics page with charts using Recharts, a responsive design, real-time budget tracking with visual progress indicators and form validation with clear, user friendly error messages.
I used context API for global authentication state management and wrote a few tests using Vitest and React testing library to ensure component reliability.

The main challenges I faced were implementing component tests, getting the charts to work and form validation to work correctly.

Setting up and writing meaningful tests for React components was a bit difficult. I needed to understand how to use React Testing Library and that took a bit of time but I was able to mostly understand how it works and use them. I started by writing tests incrementally, starting with simple rendering tests and building up to more complex user interaction tests. I ended up with 16 tests covering component rendering, form validation, and authentication flow. Integrating Recharts for the analytics page was also a hassle since I had never done that before when I built something. I had to fetch expense data from the API, aggregate it by category and time period, handle edge cases like empty datasets, and present it in three different chart types (bar, pie, and line charts). I was able to make it work by creating a dedicated Analytics component that processes the data client-side, calculates totals and percentages for each category, and dynamically renders the appropriate chart based on user selection. I also added proper loading states and error handling for when data fails to load. Implementing client-side validation for expenses and trips required careful attention to business rules. Expense dates must fall within trip dates, multi-day expenses like hotels need end dates that come after start dates, and all dates must be validated before submission. I solved this by creating reusable validation functions that check these constraints and provide clear error messages, making the interface intuitive and preventing invalid data from reaching the backend.

I am happy with how the app turned out for this assessment. I think I could have done a bit more if I had started a bit earlier and had a less jam-packed semester but I am still proud of the progress I made and the knowledge I gained from building this app. I think a fair assessment would be around the high end of level 1 and beginning of level 2.